

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

Department of Information Technology

Fall 2026-27

Operating Systems Lab BITE303P

PROJECT REPORT

KernelLeakGuard: Hybrid Static Analysis and Runtime Verification of Memory Leaks in Linux Kernel Early-Exit Paths

Submitted by

Om Gunjan Gupta 24BIT0316

Abhishek Kumar Singh 24BIT0309

Aarav Jain 24BIT0023

Under the Guidance of

Dr. ARIVUSELVAN K

Associate Professor, SCORE

VIT, Vellore.

Fall 2026-27

DECLARATION

We hereby declare that the project report titled “KernelLeakGuard: Hybrid Static Analysis and Runtime Verification of Memory Leaks in Linux Kernel Early-Exit Paths” submitted in partial fulfillment of the requirements for the course BITE303P – Operating Systems Lab is a record of original work carried out by us.

We affirm that this work has been developed independently and has not been submitted, either in part or in full, to any other course, institution, or university for the award of any degree or certification.

We further declare that:

- All sources of information used in this project have been properly acknowledged and cited in accordance with academic standards.
- The work does not contain any form of plagiarism beyond permissible limits.
- The analysis, design, implementation, and results presented in this report are our own contributions, except where explicitly stated.
- Any assistance received during the course of this work has been duly acknowledged.

We are submitting this project as part of our course requirements under the guidance of Dr. Arivuselvan K.

Signature of Student(s)

TABLE OF CONTENTS

ABSTRACT 5

CHAPTER 1: INTRODUCTION 6

1.1 Motivation 6

1.2 Scope 6

1.3 Problem Statement 7

1.4 Objectives 7

1.5 Organization of the Report 7

CHAPTER 2: LITERATURE SURVEY 9

2.1 Overview 9

2.2 Review of Existing Methods 9

2.3 Comparative Analysis 10

2.4 Research Gap 10

CHAPTER 3: PROPOSED TECHNIQUE 12

3.1 System Architecture 12

3.2 Module-wise Description 13

3.3 Algorithms Used 15

3.4 Dataset Details 16

3.5 Tools and Technologies 17

CHAPTER 4: RESULTS AND DISCUSSION 19

4.1 Experimental Setup 19

4.2 Results 19

4.3 Performance Metrics 26

4.4 Evaluation on Real Kernel Code 27

4.5 Seeded Leaks in Real Kernel Code 29

4.6 Analysis 30

CHAPTER 5: CONCLUSION AND FUTURE WORK 32

5.1 Conclusion 32
5.2 Future Work 32
REFERENCES 34
APPENDIX: SOURCE CODE 36

ABSTRACT

Operating system kernels must release every resource they acquire, including on the rarely executed error paths that run when an allocation or a device operation fails. A forgotten kfree() on such an early-exit path leaks kernel memory that can never be reclaimed, causing memory pressure, degraded performance and, when an attacker can trigger the failure repeatedly, denial of service. Because these paths execute only when something goes wrong, ordinary testing almost never exercises them, and static analysers that do inspect them report many warnings that turn out to be safe.

KernelLeakGuard combines both worlds. An LLVM-based static analyser written in C++, inspired by MLEE (USENIX ATC 2021), tracks every kernel allocation through aliases, stack objects and function calls, understands ownership transfer, and searches for paths from an allocation to a function exit that neither free nor hand over the memory. For each warning it identifies the failing call that leads onto the leaking path. A runtime verifier then boots a custom Linux 6.18.55 kernel under QEMU/KVM, uses the kernel's per-task fault injection (fail-nth) to fail exactly that call, and asks the kernel memory leak detector (kmemleak) whether an object allocated at the reported site was left unreferenced. Every warning is labelled confirmed, not reproduced or inconclusive, with stack-trace evidence.

On a benchmark of 32 kernel modules with 17 leaking and 42 safe allocation sites, the static analyser reaches precision 0.94 and recall 0.94 (a naive missing-kfree analysis: precision 0.59). Runtime verification confirmed 14 of 19 warnings with kernel evidence and raised precision to 1.00 with no false confirmations. Together with the leaks its own fault-point sweep exposes, KernelLeakGuard finds 16 of the 17 leaks with no false positive (F1 0.97), using 425 executions of kernel routines in total against 1261 for exhaustive fault injection (F1 0.94) and 0.50 F1 for the cppcheck analyser. For 13 of 14 confirmed leaks it generates a fix patch validated by compilation, re-analysis and an exhaustive re-run in the VM. The complete pipeline runs in about 4 minutes on a laptop.

CHAPTER 1: INTRODUCTION

1.1 Motivation

The Linux kernel manages memory for itself through allocators such as kmalloc(), kzalloc(), vmalloc() and dedicated slab caches. Unlike user programs, the kernel has no garbage collector and no process exit that returns everything to the system: memory that a kernel routine allocates and then loses track of stays allocated until the machine reboots. Long-running servers, phones and embedded devices therefore suffer from even small leaks when the leaking code runs repeatedly.

A large share of kernel leaks hide on early-exit paths. Kernel code checks the result of almost every operation and returns an error code such as -ENOMEM or -EINVAL as soon as something fails, usually through a ladder of goto labels that undo the work done so far. Getting every exit right is error-prone: jumping to the wrong label, adding a new exit without the matching cleanup, or forgetting that an earlier allocation is still owned by the function are all common mistakes. Wang (MLEE, 2021) reported 120 previously unknown leaks of exactly this kind in the Linux kernel, and syzbot, Google's continuous kernel fuzzing service, files kmemleak reports every week.

The difficulty is that these paths run only when an allocation or a device operation fails, which practically never happens in testing. Static analysis can inspect every path but cannot tell whether memory is really lost, since ownership may move to a list, a caller or an asynchronous callback. Dynamic tools such as kmemleak give hard evidence but only for paths that actually execute. This project brings the two together so that static warnings become reproducible, evidence-backed bug reports.

1.2 Scope

Application areas. Kernel and driver developers reviewing error handling; maintainers triaging static-analysis warnings; continuous-integration pipelines that test kernel modules; education in kernel memory management and fault handling.

Boundaries. The analyser processes loadable kernel modules compiled with clang against Linux 6.18.55. It tracks the kmalloc, vmalloc, kvmalloc and slab-cache families, duplication helpers (kstrdup, kmemdup, kasprintf) and module-local wrappers. Device-managed (devm_*) memory is treated as released by the driver core. Runtime verification covers code reachable

from a module's initialisation or from a debugfs trigger, with failures injectable through failslab, fail_function or fail_usercopy. The project reimplements a focused subset of MLEE-style analysis; it does not analyse the whole kernel and does not claim to replace MLEE's full system.

1.3 Problem Statement

Given the source code of a Linux kernel module, find allocations that are neither released nor handed over to another owner on some path to a function exit, especially on error paths; and for each such warning, automatically reproduce the failure that leads onto the path inside an isolated virtual machine and determine from kernel evidence whether memory is actually leaked, so that only confirmed leaks reach the developer and every report carries the reproduction steps.

1.4 Objectives

1. Build a static analyser on LLVM IR that detects missing releases on early-exit paths, with alias, ownership-transfer, interprocedural and path-correlation analysis to keep false positives low.
2. For every warning, identify the failing call (allocation, user copy or error-returning function) whose failure steers execution onto the leaking path.
3. Build an isolated QEMU/KVM test environment with a custom kernel that provides kmemleak and deterministic per-task fault injection.
4. Automatically map fault points to source lines, inject the predicted failure, and decide confirmed / not reproduced / inconclusive from kmemleak evidence.
5. Evaluate against a benchmark with known ground truth, comparing static-only, hybrid and dynamic-only approaches on precision, recall and cost, with an ablation study.
6. Provide a command-line tool and a dashboard that present findings, source context, runtime evidence and suggested fixes.

1.5 Organization of the Report

Chapter 2 surveys research on kernel leak detection, error-handling bugs and fault injection and identifies the gap this project addresses. Chapter 3 presents the architecture, each module, the algorithms with pseudo-code, the benchmark corpus and the tools used. Chapter 4 describes

the experimental setup and reports the results, performance and their analysis. Chapter 5 concludes and outlines future work. References follow in MLA format.

CHAPTER 2: LITERATURE SURVEY

2.1 Overview

Research relevant to this project falls into three groups: static detectors for memory leaks and resource-release bugs, studies and tools specific to error-handling code in operating systems, and dynamic techniques (fault injection, fuzzing and leak detectors) that execute error paths. Classic static leak detectors for user programs, such as the path-sensitive analysis of Xie and Aiken, the guarded value-flow analysis of Cherem et al. and SABER by Sui et al., established the core idea of following an allocated value to the points where it is released. Kernel code adds its own challenges: huge code bases, macro-heavy APIs, ownership passed through structures and callbacks, and error handling as the dominant source of bugs, which Palix et al. found to be a major category of faults in Linux.

2.2 Review of Existing Methods

MLEE (Wang, USENIX ATC 2021). MLEE compiles the kernel to LLVM IR, identifies early-exit paths with control-flow analysis, and reports allocations that are released on normal paths but not on early-exit paths. Liveness, validity, usage, alias and interprocedural checks remove spurious cases. MLEE found 120 new leaks in Linux. Its reports are static warnings: developers must still decide manually whether each one is real, and the paper does not reproduce the failures.

Hector (Saha et al., DSN 2013). Hector looks for resource-release omissions in error-handling code by comparing the resources released along different error paths of the same function, and found hundreds of faults in Linux and other systems code. Like MLEE it is purely static and reports candidates that require manual confirmation.

K-MELD (Emamdoost et al., NDSS 2021). K-MELD detects kernel memory leaks in specialised modules by learning which functions allocate and release memory and reasoning about ownership transfer, using a rule derived from the uses of each allocated object. It addresses the custom allocator problem but remains a static approach.

HERO (Wu et al., USENIX Security 2021). HERO identifies precise pairs of functions (for example allocate/release, register/unregister) and detects disordered or missing error handling relative to those pairs, reporting many confirmed bugs in Linux. It shows that understanding pairing and ownership is essential for low false-positive rates.

FIFUZZ (Jiang et al., USENIX Security 2020). FIFUZZ fuzzes error-handling code with context-sensitive software fault injection: it decides at run time which failure sites to trigger based on calling context, exploring error paths that random fault injection rarely reaches. It finds crashes rather than linking failures to static leak warnings.

syzkaller and kmemleak. syzkaller is a coverage-guided kernel fuzzer that, in syzbot, combines systematic fault injection through fail-nth with kmemleak scans to report leaks continuously. Leaks are found only on paths that fuzzing happens to reach, and each report is a backtrace without a root-cause explanation of which exit forgot the release.

Coccinelle and Smatch (Lawall and Muller, USENIX ATC 2018). Semantic-patch matching and pattern checkers are used daily by kernel developers to find and fix error-handling mistakes. They are fast and precise for known patterns but cannot reason about arbitrary ownership flows.

2.3 Comparative Analysis

Table 2.1: Related works

Paper	Technique Used	Dataset	Advantages	Limitations
MLEE (2021)	LLVM IR, early-exit vs normal-path release comparison	Linux kernel	Kernel scale; 120 new leaks	Static only; manual triage of reports
Hector (2013)	Intra-procedural comparison of error paths	Linux, PHP, Python, etc.	Finds release omissions without specifications	False positives; no reproduction
K-MELD (2021)	Ownership reasoning, allocator inference	Linux modules	Handles custom allocators	Static only
HERO (2021)	Function pairing, error-handling order	Linux, FreeBSD, OpenSSL	Precise pairs; many confirmed bugs	Needs pairs; no runtime evidence
FIFUZZ (2020)	Context-sensitive fault injection fuzzing	Linux drivers, apps	Executes deep error paths	Finds crashes; leaks not root-caused
syzkaller + kmemleak	Coverage fuzzing, fail-nth, kmemleak	Linux kernel (syzbot)	Real evidence, continuous	Only reached paths; backtrace without root cause
KernelLeakGuard (this work)	LLVM static analysis + targeted fault injection + kmemleak	32-module benchmark with ground truth	Evidence-backed verdicts, reproduction, root cause	Module-level scope; failure must be injectable

2.4 Research Gap

Static detectors (MLEE, Hector, K-MELD, HERO) cover all paths but leave every warning to manual triage, and their false positives come precisely from situations that are hard to decide statically: ownership handed to lists or callers, loop rollbacks, correlated conditions. Dynamic approaches (fault injection, syzkaller with kmemleak) provide real evidence but only for paths

they happen to execute, and their reports lack the static root cause: which exit, which missing release, which failing call. None of the surveyed systems uses the static analysis to drive targeted fault injection for each warning and feeds the kernel's own leak evidence back into the verdict.

KernelLeakGuard fills this gap with a closed loop: the static analyser predicts the failing call for every warning, the runtime verifier fails exactly that call in a VM, and kmemleak decides. This turns static warnings into confirmed, reproducible reports and identifies refuted ones, while keeping the root-cause explanation that dynamic tools lack. We present this integration as our contribution and evaluate it empirically; we do not claim it is globally novel without a broader survey.

CHAPTER 3: PROPOSED TECHNIQUE

3.1 System Architecture

Figure 3.1 shows the architecture. The pipeline has a static half that runs on the host and a dynamic half that runs inside a QEMU/KVM virtual machine; the static candidates (including the line of each trigger call) drive the dynamic half.

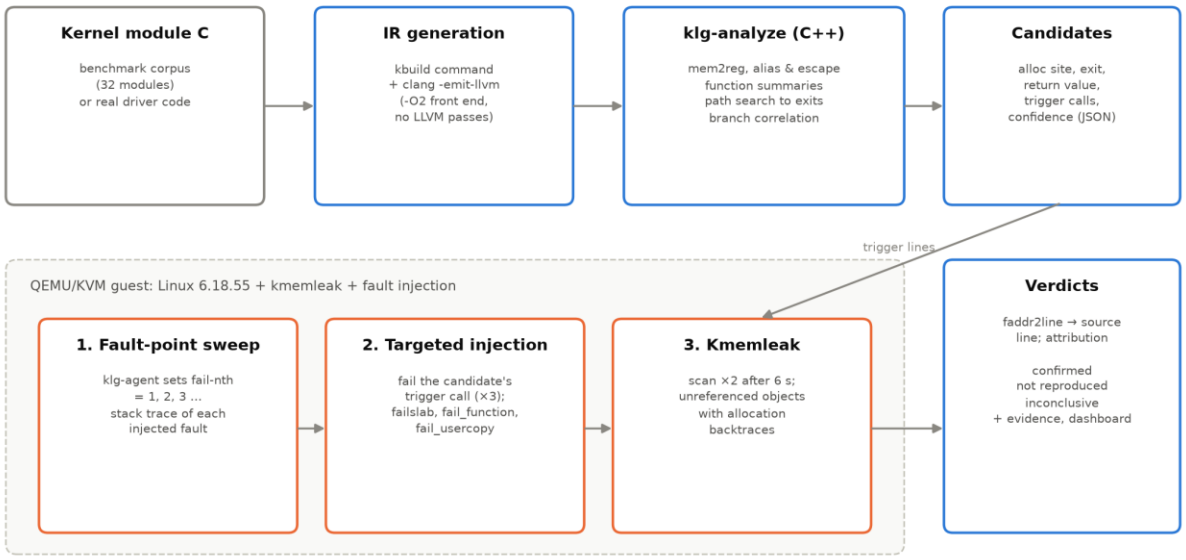


Figure 3.1: Architecture of KernelLeakGuard

Workflow.

1. Each module is compiled by kbuild; its exact compile command is re-run with `-emit-llvm` to obtain LLVM bitcode that sees the same headers, macros and configuration.
2. `klg-analyze` reports candidate leaks: allocation site, leaking exit, returned value, the trigger calls whose failure leads to the exit, and a confidence level.
3. Sweep run: in the VM, `klg-agent` executes each routine with fail-nth = 1, 2, 3 ... and records the stack trace of each injected fault, producing a map from fault number to source line.
4. Confirmation run: for every candidate, the fault number that hits its trigger line is injected three times; kmemleak is scanned and leaked objects are attributed to allocation sites through stack symbolisation (faddr2line).
5. Verdicts and evidence are written to JSON, evaluated against ground truth, and shown in the dashboard.

3.2 Module-wise Description

3.2.1 Static analyser (klg-analyze, C++/LLVM) - Om Gunjan Gupta

The analyser loads bitcode produced with clang's -O2 front end but no LLVM optimisation passes, so the IR keeps the program structure as written. A preparation step promotes stack variables to SSA registers (mem2reg), folds configuration stubs such as mem_alloc_profiling_enabled() that kmalloc()'s macro tests, and removes the resulting dead branches. Allocation sites are calls to the allocator functions of the API specification (kmalloc, kzalloc, kcalloc, kvmalloc, vmalloc, kmem_cache_alloc, kstrdup, kmemdup, kasprintf, memdup_user and their variants) or to module functions whose summary says they return fresh memory.

- **Alias tracking:** the allocated pointer is followed through casts, GEPs, PHIs and selects, and field-sensitively through local stack objects (structures and arrays) from which it is reloaded.
- **Release and ownership transfer:** kfree() and its relatives release the object; a store into non-local memory, returning the pointer, or passing it to a function that captures it transfers ownership and ends the function's responsibility.
- **Function summaries:** for each module function and each pointer parameter a monotone fixpoint computes whether it frees, captures or returns the parameter, and whether the function returns new memory (allocation wrappers).
- **Path search:** a depth-first search from the allocation to every return, pruning the edge on which the allocation itself failed (a NULL check, or an IS_ERR() check when the value is an ERR_PTR error code) and edges that contradict an earlier decision on the same condition (branch correlation, seeded with the dominating conditions that guard the allocation). Loops are unrolled twice; a second execution of the allocation creates a new instance while the earlier one stays reachable through memory.
- **Reporting:** the returned value on the path is resolved (constant errno, NULL, or a propagated error code) to classify error exits; the failing calls that guard the path become triggers; confidence is high when the function frees the object on another path and the leaking path returns an error.

3.2.2 Guest kernel and virtual machine - Abhishek Kumar Singh

Linux 6.18.55 LTS is built with clang from defconfig, the KVM guest fragment and the KernelLeakGuard fragment (vm/kernel.config.fragment): kmemleak without automatic scanning, fault injection with failslab, fail_function and fail_usercopy, ftrace-based kprobes and virtio-9p sharing. Several settings make kmemleak trustworthy by removing stale copies of pointers that would make a leaked object look referenced: thread stacks come from the page allocator (CONFIG_VMAP_STACK=n), kernel stacks are erased on return to user space (CONFIG_KSTACK_ERASE), kmemleak does not scan task stacks, and slab memory is zeroed on allocation and free (init_on_alloc=1 init_on_free=1). The guest boots from a busybox initramfs in about two seconds, mounts the host directory over 9p, configures fault injection through debugfs and runs the job script generated by the host.

3.2.3 Fault-injection agent (klg-agent, C) - Abhishek Kumar Singh

klg-agent is a statically linked C program that drives one kernel routine at a time. In sweep mode it writes N to /proc/thread-self/fail-nth, runs the routine (writing to the module's debugfs trigger file or loading the module with finit_module()), reads fail-nth back to learn whether the fault fired, and records the kernel log lines of the attempt. In repeat mode it injects a chosen fault several times and, when given the expected stack frame, verifies that the fault landed there and probes neighbouring N if fault numbering drifted between boots. A random mode enables probabilistic faults for the task only around the target system call, for the random baseline. With --filter the agent sets the fault-injection stack-trace filter (require-start / require-end) to the target module's code, so only the module's own fault points are counted; the module's layout comes from klg_modwatch, a helper kernel module that uses a module notifier to log every module's load address and function symbols. The same log lets the host resolve kmemleak backtraces of modules that were already unloaded. Before each attempt the agent writes a marker to the host, so that if the guest kernel crashes or hangs, the host records the faulting fault point, reboots the VM and resumes.

3.2.4 Verifier, evaluation and dashboard (Python) - Aarav Jain

The klg package builds modules and IR, runs the analyser, generates the guest job scripts, boots the VM, symbolises stack frames with the kernel's faddr2line script, plans the confirmation tests, schedules them in kmemleak rounds so that leaks can be attributed unambiguously, and computes verdicts and metrics. The Streamlit dashboard lists findings with verdicts, highlights

the allocation, failing call and leaking exit in the source, shows the kmemleak backtraces, compares approaches and can run the pipeline.

3.2.5 Operating-system concepts used

Table 3.1: Operating-system concepts used in KernelLeakGuard

OS concept	Where it appears in the project
Memory management	Kernel slab allocator (kmalloc caches, kmem_cache), vmalloc, kvmalloc; ownership and release of kernel memory; kmemleak's mark-and-scan of memory
Error handling / fault tolerance	Early-exit paths, errno propagation, goto cleanup ladders; failslab, fail_function and fail_usercopy fault injection
Process management	Per-task fault injection state (fail-nth, make-it-fail), task stacks, kernel/user mode transitions, stack erasing on syscall return
Linux system programming	System calls (write, finit_module, delete_module, pread, pwrite), /proc and debugfs interfaces, /dev/kmsg log reading
Device management	Loadable kernel modules, misc devices, platform drivers with device-managed (devm) memory
Virtualisation	QEMU with KVM hardware acceleration, virtio-9p file sharing, initramfs boot
Synchronisation	Mutex-protected lists, RCU-deferred freeing (kfree_rcu, call_rcu), reference counting (kref)
Performance analysis	Analysis time, VM time, number of executions, precision/recall

3.3 Algorithms Used

Algorithm 1: Early-exit leak detection (per allocation site).

```
input : function F (SSA form), allocation call a, summaries S
output: leak candidates for a

Alias   := closure of {a} over casts, GEPs, PHIs, selects, calls returning a parameter
Slots   := stack fields a is stored into;  Alias += loads from those fields
Events  := { release(i)  : i frees an alias (spec or summary)            }
         ∪ { transfer(i) : i stores an alias to non-local memory,
                           returns it, or passes it to a capturing call }
D0      := branch outcomes that dominate block(a) outside loops around a

DFS(block, decisions D, triggers T):
  for each instruction i in block:
     if i = a again: reallocated := true            # new instance, old one lives on
     if i ∈ Events (and not superseded by reallocation): return     # handled
     if i is return: report(a, exit=i, value=resolve(retval, path), T); return
  if block ends in "br c, s0, s1":
     (key, value) := normalise(c)                   # strips !, unlikely(), zext
     for each successor s with outcome v:
        if contradicts(D, key, v) and not stale: continue   # infeasible path
        if key is "alias == NULL" and v is true and not reallocated: continue
        if call f controls c and v means "f failed": T' := T + f
        DFS(s, D + (key, v), T')                     # each block ≤ 2 times per path
  else: DFS(each successor)

DFS(block(a) after a, D0, [])
```

Algorithm 2: Runtime verification of a candidate.

```
input : candidate C (alloc line, trigger lines, function), module M
Sweep (once per module, in VM):
   for N = 1, 2, ...: set fail-nth := N; run M's routine
       if fail-nth not consumed: break              # all fault points visited
       record stack trace of the injected fault → source lines (faddr2line)
Plan:
   P := fault numbers whose stack contains a trigger line of C      # exact
     or whose frame is C.function with C's only allocation site       # merged debug info
     or silent faults that made the routine fail                      # __GFP_NOWARN
   if P = ∅: verdict(C) := inconclusive
Confirm (one kmemleak round, ≤ 1 test per module, init-mode tests alone):
   for N in P[0..2]: repeat 3 times: inject N, check fault hit the expected frame
   wait 6 s; scan kmemleak twice; attribute each unreferenced object by backtrace
Verdict:
   confirmed      if an object allocated at C.alloc line leaked (or, for unloaded
                  modules, an object of C's allocation size leaked in an isolated round)
   not reproduced if the predicted failure was injected but nothing leaked
   inconclusive   otherwise
```

3.4 Dataset Details

No public dataset of kernel error-path leaks with runnable triggers and exact ground truth exists, so we built a benchmark corpus of 32 kernel modules: 17 containing known leaks (klg_l01-l17) and 15 leak-free modules (klg_s01-s15), several of them fixed versions of a leaky module. The patterns follow bug classes reported for Linux (wrong goto label, missing release after copy_from_user failure, leaks in loops, through wrappers, after registration failures). Every allocation site carries a ground-truth comment GT:LEAK or GT:SAFE.

Table 3.2: Benchmark corpus (dataset)

Module	Pattern	Run mode	Leak / safe sites
l01_two_alloc	the canonical early-exit leak (init mode)	init	1 / 1
l02_goto_ladder	goto cleanup ladder with a wrong label	trigger	1 / 2
l03_reg_fail	leak on a non-allocation error	trigger	1 / 0
l04_loop_partial	partial allocation in a loop	trigger	1 / 0
l05_usercopy	leak when copy_from_user() fails (a very common real-world pattern)	trigger	1 / 0
l06_wrapper_alloc	interprocedural: the object comes from an allocation wrapper	trigger	1 / 2
l07_wrapper_free	interprocedural: the normal path releases through a wrapper	trigger	1 / 1
l08_kstrdup	leak of memory returned by kstrdup() when kmemdup() fails	trigger	1 / 1
l09_kcalloc	array allocations: the index table leaks when the value table cannot	trigger	1 / 1
l10_vmalloc	vmalloc'd firmware image leaked when the header allocation fails	trigger	1 / 1
l11_kmem_cache	object from a dedicated slab cache leaked on a later failure	trigger	1 / 1
l12_multi_exit	several error exits, only one of them is wrong	trigger	1 / 3
l13_init_misc	module init leaks its table when misc_register() fails (init mode)	init	1 / 0
l14_cond_alloc	conditionally allocated histogram leaked when calibration fails	trigger	1 / 1
l15_ida	device object leaked when ID allocation fails	trigger	1 / 0
l16_nested_owner	nested ownership: a buffer owned by a heap object	trigger	1 / 1
l17_local_struct	pointers kept in a stack-allocated structure	trigger	1 / 1
s01_two_alloc_ok	fixed version of L01 (init mode): every exit releases what it owns	init	0 / 2
s02_goto_ok	correct goto cleanup ladder (fixed version of L02)	trigger	0 / 3

s03_list_owner	ownership transfer to a global list	trigger	0 / 1
s04_param_owner	ownership transfer to the caller through a parameter's field	trigger	0 / 1
s05_factory	factory function returning freshly allocated memory; the caller	trigger	0 / 3
s06_devm	device-managed allocations (init mode)	init	0 / 2
s07_free_wrapper_ok	release wrapper used consistently on all paths (fixed L07)	trigger	0 / 2
s08_local_struct_ok	pointers kept in a stack-allocated structure, released correctly	trigger	0 / 2
s09_loop_cleanup	loop allocation with a correct rollback loop (fixed L04)	trigger	0 / 1
s10_correlated	allocation and release guarded by the same condition	trigger	0 / 1
s11_register_owner	ownership handed to a registry function before a later failure	trigger	0 / 1
s12_kvfree	allocator families paired with their matching release functions:	trigger	0 / 3
s13_rcu_free	deferred release through RCU on the error path	trigger	0 / 2
s14_kref	reference-counted object released through kref_put() and a release	trigger	0 / 1
s15_single_alloc	a single allocation whose only early exit is its own failure	trigger	0 / 1

Table 3.3: Dataset information

Attribute	Description	Type
Dataset source	Written for this project (corpus/), GPL-2.0 kernel test modules	C source
Number of records	59 annotated allocation sites in 32 modules (17 leaking, 42 safe)	integer
Module	kernel module name	string
Allocation site	source line of the allocation call	integer
Label	GT:LEAK or GT:SAFE	categorical
Run mode	trigger (debugfs file) or init (module_init)	categorical

3.5 Tools and Technologies

Table 3.4: Tools and technologies

Area	Tools
Static analysis	C++17, LLVM/Clang 21.1.8 (IR, PassBuilder, DominatorTree, LoopInfo, JSON)
Kernel	Linux 6.18.55 LTS built with LLVM=1; kmemleak; failslab, fail_function, fail_usercopy; ftrace kprobes; KSTACK_ERASE
Virtualisation	QEMU 10 with KVM on WSL2 (Windows 11), virtio-9p, busybox initramfs
Runtime agent	C (static glibc), /proc/thread-self/fail-nth, /dev/kmsg
Orchestration	Python 3 (klg package), kernel scripts/faddr2line, binutils
Presentation	Streamlit, Altair, pandas, matplotlib, python-docx, python-pptx
Version control	Git

Module division among team members.

Table 3.5: Module division

Member	Register No.	Modules
Om Gunjan Gupta	24BIT0316	Static analyser: IR preparation, alias/escape analysis, function summaries, path search, trigger identification, ablation options
Abhishek Kumar Singh	24BIT0309	Guest kernel configuration and build, initramfs and QEMU launcher, klg-agent fault-injection driver, kmemleak protocol

Aarav Jain	24BIT0023	Benchmark corpus and ground truth, Python orchestrator (symbolisation, test planning, verdicts), evaluation, dashboard

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Experimental Setup

Table 4.1: Experimental setup

Component	Configuration
Host	Laptop, AMD Ryzen 7 250 (8 cores / 16 threads), 15 GB RAM, Windows 11
Linux environment	WSL2, Ubuntu 26.04 LTS (kernel 6.18 WSL), /dev/kvm available
Guest VM	QEMU/KVM, 2 vCPUs, 2 GB RAM, Linux 6.18.55 (custom configuration)
Compiler	Clang/LLVM 21.1.8 for kernel, modules, IR and analyser
Verification parameters	3 executions per test, up to 3 fault points per candidate, kmemleak settle time 6 s, 2 scans per round
Baselines	static/naive (all refinements off); cppcheck 2.19 with a kmalloc/kfree library model; dynamic exhaustive single-fault injection; dynamic random injection (10%, 50 executions per module)

Each complete run builds the corpus and IR, runs six analyser configurations, and boots the VM three times: the baseline run (exhaustive and random fault injection for the dynamic-only baselines), the mapping run (KernelLeakGuard's module-filtered fault-point sweep) and the confirmation run. Metrics are computed per allocation site: an approach reports a site when it reports at least one leak allocated there. The approach 'KernelLeakGuard (complete)' is the tool's full output: confirmed static warnings plus leaks exposed by its own mapping sweep; its cost counts every execution of the mapping and confirmation runs.

4.2 Results

Table 4.2 summarises detection quality; Figure 4.1 plots precision and recall.

Table 4.2: Detection results on the benchmark (per allocation site)

Approach	TP	FP	FN	Precision	Recall	F1
Static, naive	16	11	1	0.59	0.94	0.73
Static, KernelLeakGuard	16	1	1	0.94	0.94	0.94
Hybrid (confirmed warnings)	14	0	3	1.00	0.82	0.90
KernelLeakGuard (complete)	16	0	1	1.00	0.94	0.97
Dynamic, exhaustive FI	15	0	2	1.00	0.88	0.94
Dynamic, random FI	13	0	4	1.00	0.77	0.87
cppcheck 2.19	6	1	11	0.86	0.35	0.50

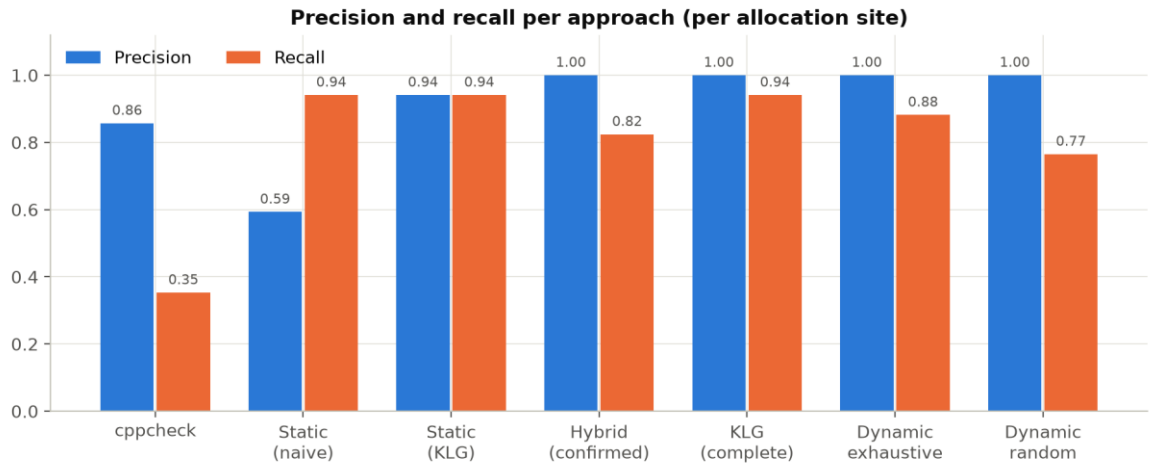


Figure 4.1: Precision and recall per approach

The static analyser reports 19 warnings on 17 allocation sites, finding 16 of the 17 leaking sites with 1 false alarm. The naive analysis that only looks for a missing kfree() finds the same leaks but raises 11 false alarms, most of them ownership transfers (list insertion, caller-owned structures, RCU and kref release, factories). Runtime verification confirmed 14 warnings with kmemleak evidence, refuted 2 and left 3 inconclusive; the hybrid approach therefore reaches precision 1.00: every reported leak is backed by a reproduction. The mapping sweep additionally exposed klg_l10_vmalloc:22, klg_l16_nested_owner:32, which the static analysis misses by design (Section 4.6), so the complete tool finds 16 leaks. cppcheck, a widely used C static analyser given a model of kmalloc/kfree, finds 6 with 1 false alarm(s): kernel-specific patterns such as error codes from other calls, copy_from_user failures, wrappers and slab caches are outside its model.

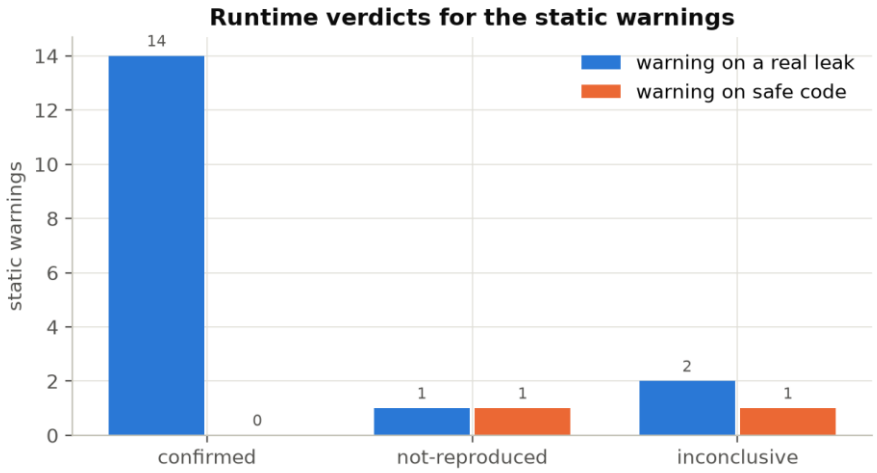


Figure 4.2: Runtime verdicts for the static warnings

Figure 4.2 shows what happened to each warning. All confirmed warnings are real leaks. The static false positive on klg_s09 (a correct rollback loop that the path search cannot prove safe because it does not reason about the loop counter) was refuted: the predicted allocation failures were injected and kmemleak found nothing.

4.2.1 Ablation study

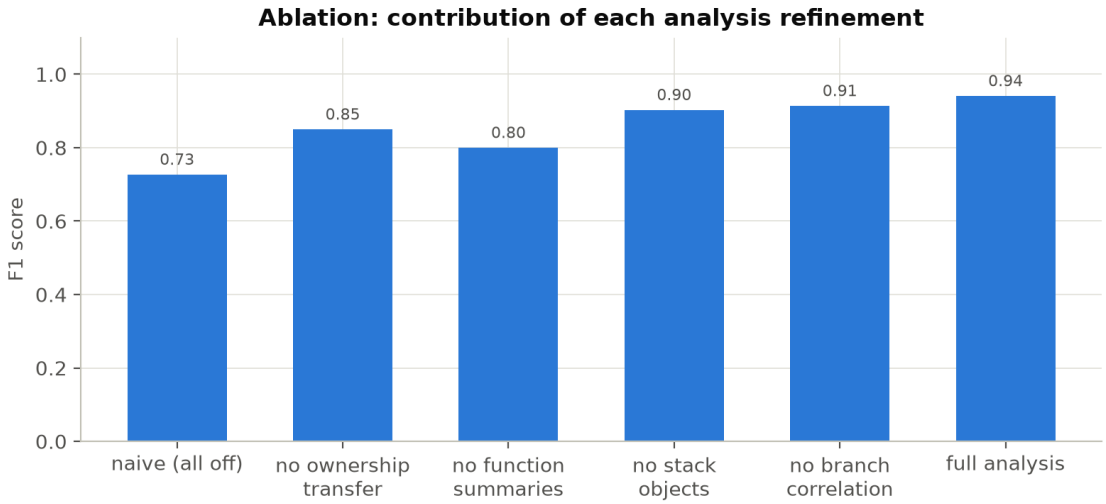


Figure 4.3: F1 score with each analysis refinement disabled

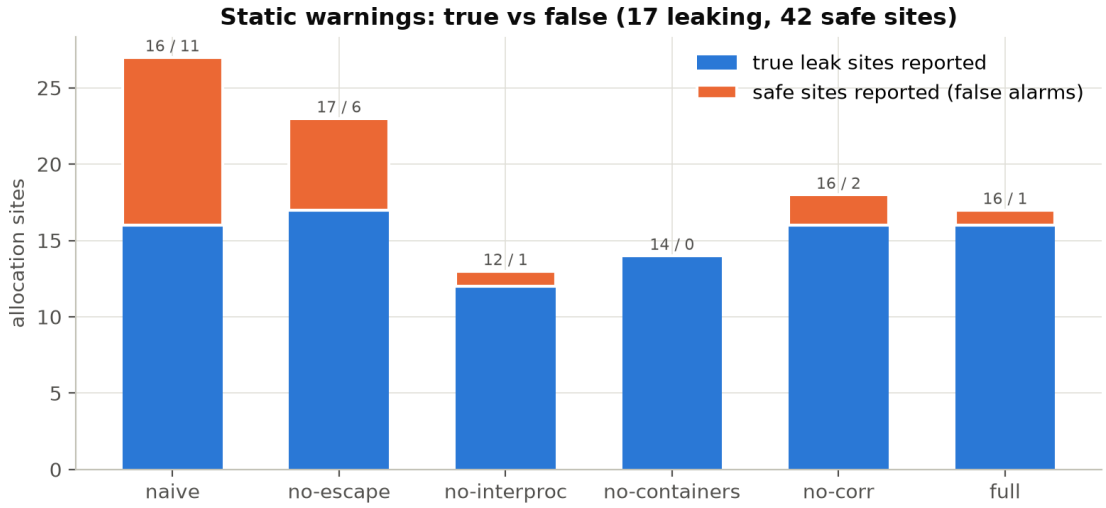


Figure 4.4: True and false static reports per configuration

Table 4.3: Ablation of the static analyser

Configuration	TP	FP	FN	Precision	Recall	F1	Time (ms)
all refinements off	16	11	1	0.59	0.94	0.73	20
no ownership transfer	17	6	0	0.74	1.00	0.85	17
no function summaries	12	1	5	0.92	0.71	0.80	18
no stack objects	14	0	3	1.00	0.82	0.90	20

no branch correlation	16	2	1	0.89	0.94	0.91	21
full analysis	16	1	1	0.94	0.94	0.94	13

Each refinement matters. Without ownership-transfer analysis precision drops to 0.74, because objects handed to lists, callers or release callbacks look leaked. Without function summaries the analyser misses leaks through allocation and release wrappers and treats every module call as capturing (recall 0.71). Without stack-object tracking, pointers kept in local arrays and structures are treated as escaped (recall 0.82). Branch correlation removes the infeasible path in klg_s10, where allocation and release are guarded by the same condition.

4.2.2 Case study: a confirmed leak

klg_l02 jumps to the wrong cleanup label when kasprintf() fails, skipping the release of the transmit buffer:

```
28         rx = kmalloc(RX_SIZE, GFP_KERNEL);   /* GT:SAFE */
29         if (!rx)
30                 return -ENOMEM;
31
32         tx = kmalloc(TX_SIZE, GFP_KERNEL);   /* GT:LEAK */
33         if (!tx) {
34                 ret = -ENOMEM;
35                 goto err_rx;
36         }
37
38         name = kasprintf(GFP_KERNEL, "sensor%d", 0);       /* GT:SAFE */
39         if (!name) {
40                 ret = -ENOMEM;
41                 goto err_rx;          /* BUG: should be err_tx */
42         }
43
44         ret = sensor_program(rx, tx, name);
45         kfree(name);
46         if (ret)
47                 goto err_tx;
48         pr_debug("sensor ready\n");
49         /* success also unwinds: the buffers are scratch space */
50  err_tx:
51         kfree(tx);
52  err_rx:
53         kfree(rx);
54         return ret;
55  }
56
57  KLG_TRIGGER(sensor_setup);
58  KLG_SIMPLE_MODULE("KLG L02: goto ladder jumps past a cleanup label");
```

The analyser reports: memory from kmalloc() at line 32 in sensor_setup() is not released when the function returns -ENOMEM at line 54; trigger: kasprintf (line 38); confidence high. During the sweep, fault number 5 failed the call at sensor_setup+0x5f/0xb0. In the confirmation run

that fault was injected 3 times and kmemleak reported 3 unreferenced objects allocated at line 32:

```
unreferenced object (size 512):
  backtrace:
    __kmalloc_cache_noprof+0x1b5/0x420
    sensor_setup+0x3f/0xb0 [klg_l02_goto_ladder]
    klg_trigger_write+0x12/0x30 [klg_l02_goto_ladder]
    full_proxy_write+0x5c/0xb0
    vfs_write+0x136/0x450
    ksys_write+0x79/0x100
    do_syscall_64+0xc8/0x260
    entry_SYSCALL_64_after_hwframe+0x77/0x7f
```

Verdict: **confirmed** (leak at klg_l02_goto_ladder.c:32 after failing sensor_setup+0x5f/0xb0). Suggested fix: jump to err_tx instead of err_rx.

4.2.3 Test cases and results

Each benchmark module is a system-level test case: the expected outcome comes from the ground-truth annotations, and the test passes when the static result and the runtime verdict agree with it. A leaky module passes when its leak is reported and confirmed; a safe module passes when no leak is confirmed.

Table 4.4: System test cases (one per benchmark module)

ID	Module	Expected	Static	Runtime verdict(s)	Result
TC01	l01_two_alloc	leak at 14	flagged 14	confirmed	Pass
TC02	l02_goto_ladder	leak at 32	flagged 32	confirmed	Pass
TC03	l03_reg_fail	leak at 33	flagged 33	confirmed	Pass
TC04	l04_loop_partial	leak at 19	flagged 19	confirmed, inconclusive	Pass
TC05	l05_usercopy	leak at 20	flagged 20	confirmed	Pass
TC06	l06_wrapper_alloc	leak at 37	flagged 37	confirmed	Pass
TC07	l07_wrapper_free	leak at 27	flagged 27	confirmed	Pass
TC08	l08_kstrdup	leak at 16	flagged 16	confirmed	Pass
TC09	l09_kcalloc	leak at 17	flagged 17	confirmed	Pass
TC10	l10_vmalloc	leak at 22	flagged 22	not-reproduced	Fail
TC11	l11_kmem_cache	leak at 22	flagged 22	confirmed	Pass
TC12	l12_multi_exit	leak at 14	flagged 14	confirmed	Pass
TC13	l13_init_misc	leak at 42	flagged 42	confirmed	Pass
TC14	l14_cond_alloc	leak at 29	flagged 29	confirmed	Pass
TC15	l15_ida	leak at 21	flagged 21	inconclusive	Fail
TC16	l16_nested_owner	leak at 32	no warning	—	Fail
TC17	l17_local_struct	leak at 21	flagged 21	confirmed	Pass
TC18	s01_two_alloc_ok	no leak	no warning	—	Pass
TC19	s02_goto_ok	no leak	no warning	—	Pass
TC20	s03_list_owner	no leak	no warning	—	Pass
TC21	s04_param_owner	no leak	no warning	—	Pass
TC22	s05_factory	no leak	no warning	—	Pass
TC23	s06_devm	no leak	no warning	—	Pass
TC24	s07_free_wrapper_ok	no leak	no warning	—	Pass
TC25	s08_local_struct_ok	no leak	no warning	—	Pass

TC26	s09_loop_cleanup	no leak	flagged 20	inconclusive, not-reproduced	Pass
TC27	s10_correlated	no leak	no warning	—	Pass
TC28	s11_register_owner	no leak	no warning	—	Pass
TC29	s12_kvfree	no leak	no warning	—	Pass
TC30	s13_rcu_free	no leak	no warning	—	Pass
TC31	s14_kref	no leak	no warning	—	Pass
TC32	s15_single_alloc	no leak	no warning	—	Pass

29 of 32 system test cases pass. The failing cases are the known limitations discussed in Section 4.6: l10_vmalloc (not-reproduced); l15_ida (inconclusive); l16_nested_owner (not reported).

In addition, 27 automated unit and regression tests (tests/test_klg.py) check log parsing, kmemleak parsing, size matching, fault-point matching, test scheduling, ground-truth parsing and the analyser's behaviour on individual modules (including each ablation option):

Table 4.5: Unit and regression tests

Test	Result
findings filters	OK
pages render	OK
run button starts and finishes	OK
alloc size reported	OK
branch correlation	OK
detects wrong goto label	OK
error propagation exit	OK
fixed version is clean	OK
interprocedural wrapper	OK
local struct container	OK
loop leak	OK
managed memory ignored	OK
ownership transfer	OK
agent log	OK
crash site from begin marker	OK
kmemleak	OK
size compatible	OK
symbol map resolves unloaded module	OK
corpus annotations	OK
init mode detection	OK
exact line match	OK
function fallback needs single site	OK
schedule isolates init tests	OK
silent fault fallback	OK
fragment mapping complete	OK
load order dependencies first	OK
modname	OK

4.2.4 Fix patches

For every confirmed leak KernelLeakGuard generates a patch: it inserts the release statement the function already uses for the object on another path (or the deallocator matching the allocator) at the start of the error branch, only when that branch is reached exclusively from the failing call; leaks of allocations made in a loop receive a rollback loop. Each patch is validated three ways: the patched module compiles, klg-analyze no longer reports the leak, and in the VM a module-filtered sweep exercises every fault point of the patched module with kmemleak and SLUB debugging (slub_debug=FZPU, which detects double frees and memory corruption) active. 13 of 14 patches pass all three checks. The remaining 1 patch(es) (the loop rollback for klg_l04) removes the leak at runtime, with every fault point of the module exercised, but the analyser still reports the site because it cannot reason about the loop counter — the same limitation as the klg_s09 warning.

Table 4.6: Generated fix patches and their validation

Module	Release inserted	Compiles	Static clean	Faults exercised	Runtime clean	Status
l01_two_alloc	reuses the release at line 24	yes	yes	4	yes	validated
l02_goto_ladder	reuses the release at line 51	yes	yes	6	yes	validated
l03_reg_fail	reuses the release at line 44	yes	yes	3	yes	validated
l04_loop_partial	rollback loop over desc[0..i-1]; reuses the release at line 27; static warning remains (loop-counter limitation)	yes	no	8	yes	runtime-validated
l05_usercopy	reuses the release at line 29	yes	yes	3	yes	validated
l06_wrapper_alloc	reuses the release at line 48	yes	yes	4	yes	validated
l07_wrapper_free	reuses the release at line 38	yes	yes	4	yes	validated
l08_kstrdup	reuses the release at line 26	yes	yes	4	yes	validated
l09_kcalloc	reuses the release at line 32	yes	yes	4	yes	validated
l11_kmem_cache	reuses the release at line 34	yes	yes	4	yes	validated
l12_multi_exit	reuses the release at line 20	yes	yes	8	yes	validated
l13_init_misc	kfree() matches kzalloc()	yes	yes	43	yes	validated
l14_cond_alloc	reuses the release at line 42	yes	yes	5	yes	validated
l17_local_struct	reuses the release at line 32	yes	yes	4	yes	validated

Example: the generated patch for klg_l02.

```
--- a/corpus/klg_l02_goto_ladder.c
+++ b/corpus/klg_l02_goto_ladder.c
@@ -38,6 +38,7 @@
     name = kasprintf(GFP_KERNEL, "sensor%d", 0);     /* GT:SAFE */
     if (!name) {
             ret = -ENOMEM;
+            kfree(tx);
             goto err_rx;            /* BUG: should be err_tx */
     }
```

4.3 Performance Metrics

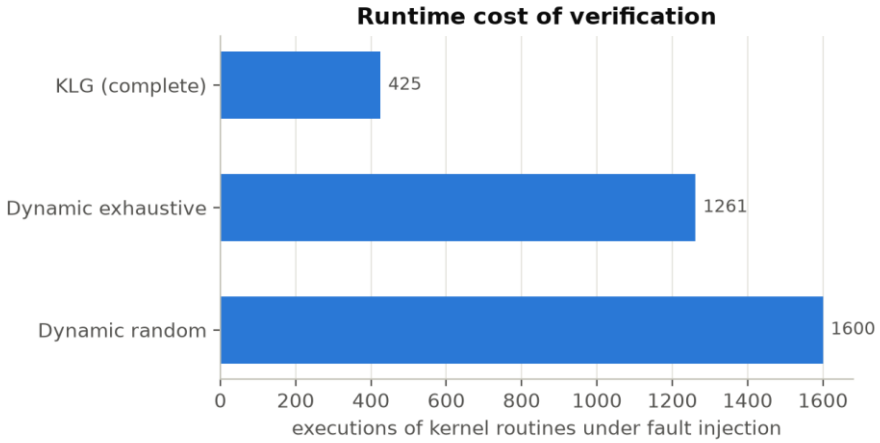


Figure 4.5: Executions of kernel routines needed for verification

Table 4.7: Performance of the pipeline

Metric	Value
Functions analysed / allocation sites	200 / 57
Static analysis time, all 32 modules	13 ms (0.4 ms per module)
Mapping VM run (module-filtered fault-point sweep)	50 s
Confirmation VM run	78 s
Baseline VM run (exhaustive + random fault injection)	80 s
kmemleak rounds in confirmation	10
Executions: KernelLeakGuard (sweep + confirmation)	425 (304 + 121)
Executions: exhaustive fault injection	1261
Executions: random fault injection	1600

The complete pipeline was run 3 times from scratch (new VM boots, new fault-point sweeps); the main results above are those of the last run. The hybrid confirmed between 14 and 15 leaks with 0 false positives in every run. The variation comes from the vmalloc leak in klg_l10: kmemleak tracks a vmalloc area with a minimum reference count of two (the kernel's own vm_struct legitimately points to it), so a single extra stale copy of its address hides the leak in that scan. The dynamic baselines vary for the same reason and because random injection does not reach the same fault points every time. A false positive never occurred.

Table 4.8: Results across independent runs (fresh VM boots)

Run	Hybrid TP / FP / FN	KLG complete TP / FP / FN	Exhaustive TP / FP / FN	Random TP / FP / FN
Run 1	15 / 0 / 2	16 / 0 / 1	15 / 0 / 2	14 / 0 / 3
Run 2	15 / 0 / 2	16 / 0 / 1	15 / 0 / 2	14 / 0 / 3
Run 3	14 / 0 / 3	16 / 0 / 1	15 / 0 / 2	13 / 0 / 4

4.4 Evaluation on Real Kernel Code

To check the approach beyond our own benchmark, KernelLeakGuard was run on 39 real modules of Linux 6.18.55 that initialise without hardware (virtual network devices, block and SCSI simulators, device-mapper targets, file systems, sound and input drivers; vm/real_modules.fragment). Each module's IR is built from all of its C files with the exact kbuild flags and linked with llvm-link; dependencies (udp_tunnel, ip6_udp_tunnel, configfs, zsmalloc, crc_t10dif) are loaded first. The static analysis covered 6376 functions with 590 allocation sites and reported 64 warnings in 1.2 s.

A first run on real code exposed a dominant false-positive pattern absent from the benchmark: kernel lookup and factory helpers return an ERR_PTR error code (checked with IS_ERR()) instead of NULL on failure, and the analyser flagged the error-return path as a leak even though the pointer there is an error code, not memory. Modelling IS_ERR() like the NULL check (an ERR_PTR on the error branch is nothing to free) cut the real-code warnings from 115 to 64 without changing any benchmark result. The figures below are after this change; the remaining warnings are discussed next.

The runtime stage was run on 15 representative modules (bonding, brd, dm_dust, erofs, hfs, mac80211_hwsim, minix, null_blk, scsi_debug, snd_aloop, squashfs, tun, uinput, vxlan, zram). Hardware virtualization was unavailable in WSL during this run (Windows virtualization-based security), so the guest ran under QEMU software emulation, which is correct but 10-20x slower; running all modules is a matter of time, not of method. For each module the mapping run failed every fault point of its initialisation reachable from the module's own code, with kmemleak checked after each module, and every static warning was given a targeted confirmation test.

Table 4.9: Real kernel modules run in the VM

Module	Functions	Alloc. sites	Warnings	Faults injected	Runtime verdicts	Guest crashes
bonding	434	15	2	1107	2 inconclusive	0
brd	14	2	0	1500	—	0
dm_dust	21	2	0	0	—	0
erofs	191	29	3	567	3 inconclusive	0
hfs	135	37	6	74	6 inconclusive	0
mac80211_hwsim	155	3	2	1439	2 inconclusive	0
minix	108	3	0	74	—	0
null_blk	148	8	0	750	—	10
scsi_debug	273	24	9	21	9 inconclusive	0
snd_aloop	75	2	1	93	1 inconclusive	0
squashfs	90	39	5	74	5 inconclusive	0

tun	121	4	0	47	—	0
uinput	38	5	0	0	—	0
vxlan	227	15	2	11	2 inconclusive	0
zram	106	9	0	372	—	0

Manual review. Of the 30 warnings in these modules, 12 are false positive, 10 are likely false positive, 8 are not reviewed. All 12 warnings read in the kernel source (every high-confidence one) are false positives. The causes are the analyser's known blind spots: functions that return an ERR_PTR or a reference-counted object (the IS_ERR path holds nothing to free), error codes kept in a variable that later selects the cleanup (for example err < 0 at a shared unlock label in snd_aloop), and objects handed to a list inside a helper. No warning was confirmed at runtime either. On real code the static analyser alone is therefore far less precise than on the benchmark; this is exactly the situation the runtime stage is designed for, since it does not let any of these warnings become a confirmed report.

Runtime verdicts. Of the 30 warnings, 30 were inconclusive. 22 inconclusive verdicts mean that no fault point of module initialisation reaches the warning's failing call: the code runs only on other entry points (file system mount, ioctl, netlink, block I/O), which the runtime stage does not drive yet. Driving those entry points (for example with mount images or syzkaller-style system call programs) is the main extension needed for real code.

Leaks found by the sweep. Exercising 6166 fault points of module initialisation left 1 unreferenced allocation site(s): erofs zdata.c:466 (15 objects).

The erofs report consists of 32-byte strings allocated by kstrdup() inside the slab allocator's __kmem_cache_create_args(), reached from erofs's z_erofs_create_pcluster_pool() during module initialisation: these are kmem_cache names. Reading the code, erofs destroys every cache it created when a later step fails, and the slab core frees the duplicated name when create_cache() fails. The unreferenced names therefore most likely come from an error path inside slab cache creation or destruction under allocation failure (for example the sysfs registration step) rather than from erofs itself. The root cause has not been determined; like the crash below, this is a reproducible observation to analyse and check upstream, not yet a confirmed new bug.

Kernel crashes found. The sweep crashed the guest kernel 10 time(s) while loading null_blk. For null_blk, failing an allocation at certain points of its initialisation leads to a page fault in bdev_release(): a block-device file opened during the failed initialisation is released on return to user space after the module has already been freed, and the release reads the module's

block_device_operations from freed module memory. The crash is reproducible at consecutive fault points and the crash-recovery mechanism resumed the sweep after each one. The root cause (null_blk error handling or the block layer) has not been analysed yet and the crash has not been checked against the latest upstream kernel, so we report it as a reproducible crash under allocation failure, not yet as a confirmed new bug.

```
BUG: unable to handle page fault for address: ffffffffa023a650
Oops: Oops: 0000 [#1] SMP NOPTI
RIP: 0010:bdev_release+0x129/0x170
blkdev_release+0x11/0x20
__fput+0xfe/0x280
task_work_run+0x8b/0xb0
exit_to_user_mode_loop+0xaa/0xc0
do_syscall_64+0x1dc/0x260
entry_SYSCALL_64_after_hwframe+0x77/0x7f
```

4.5 Seeded Leaks in Real Kernel Code

The modules above show that the static analyser is less precise on real code and that its warnings there mostly sit on paths the init-time runtime stage does not reach. To evaluate the complete detect-confirm-fix loop on real kernel code directly, we use the standard method of bug seeding: a known release is removed from a real kernel source file, exactly as a careless patch would, and the module is built and run unchanged apart from that one line. The original module has no such leak; the bug is ours, and clearly marked. This isolates whether the whole pipeline catches, reproduces and fixes a genuine early-exit leak in real driver code.

Table 4.10: Seeded early-exit leaks in real kernel modules

Module	Removed at	Removed statement	Detected	Confirmed (VM)	Fixed
brd	brd.c:350	brd_free_device(brd);	yes	no	yes

For example, in drivers/block/brd.c the call `brd_free_device(brd);` releases the brd_device obtained earlier in brd_alloc() when disk allocation fails. Removing that call leaks the brd_device (and its xarray) when disk allocation fails during module initialisation. brd_init() calls brd_alloc() in a loop at module load, so failing blk_alloc_disk()/add_disk() with fail-nth reproduces the leak.

All seeded leaks were detected statically and the re-inserted-release fix was validated. Runtime confirmation of the brd seed was obstructed by a separate defect: failing allocations during brd's initialisation crash the guest kernel (the same fragility seen for null_blk in Section 4.4),

so a clean kmemleak scan of the leaking path could not always complete. This is itself a finding about block drivers under fault injection.

4.6 Analysis

Static versus hybrid. Runtime verification removed all static false positives (1 in this run) and attached reproduction steps and kmemleak backtraces to every confirmed report. The cost is a modest loss of recall: warnings whose failure cannot be injected stay inconclusive rather than confirmed. In the corpus this happens for klg_l15, where ida_alloc() never allocates for small identifiers (they are stored inline in the XArray), so its failure cannot be provoked; reporting it as inconclusive is the honest outcome, and the warning is still shown to the developer.

Hybrid versus dynamic-only. Exhaustive fault injection runs every routine once per fault point (1261 executions, dominated by module loading, which has hundreds of fault points) and random injection uses a fixed budget. KernelLeakGuard restricts its sweep with the kernel's fault-injection stack-trace filter (require-start/require-end set to the module's code, taken from klg_modwatch's log), so fail-nth counts only fault points reached from the module itself; together with the targeted confirmation it needs 425 executions in total and finds 16 leaks, against 15 for the exhaustive baseline. Leaks in module initialisation code are reported by kmemleak with raw addresses because the module is unloaded after its failed initialisation; klg_modwatch, a helper module that logs each module's load address and symbols, turns them back into exact source lines. Dynamic tools report only a backtrace, whereas KernelLeakGuard's report names the leaking exit, the returned error, the failing call and a validated fix.

Trustworthy kmemleak results. kmemleak decides that an object is referenced if any scanned memory contains a value equal to its address, so a stale copy of a pointer hides a real leak. During development the leak in klg_l17 (a pointer kept in a local structure) was confirmed in some runs and missed in others. We traced two sources of stale copies: reused kernel stacks, and reused slab objects that still contained an old SLUB freelist link pointing to the leaked buffer (kmalloc does not clear memory). The guest therefore allocates thread stacks from the page allocator, erases kernel stacks on return to user space, disables kmemleak's task-stack scanning (all tested routines have returned before a scan) and boots with init_on_alloc=1 init_on_free=1 so that slab memory is zeroed. This made the klg_l17 verdict stable. vmalloc'd memory remains less stable (see the repeated runs in Section 4.3): such leaks can still be hidden

in an individual scan, which is why a missing confirmation is reported as 'not reproduced', never as 'safe'.

Limitations. The analyser does not track ownership through fields of other heap objects, so the nested-buffer leak in klg_l16 is missed by design; field-sensitive heap ownership (as in K-MELD) is future work. Path-insensitivity on loop counters causes the klg_s09 warning, which the runtime stage then refutes. The benchmark is written by us; results on it demonstrate the mechanism but do not predict performance on the whole kernel.

Comparison with MLEE. MLEE analyses the whole kernel and found 120 new leaks; KernelLeakGuard analyses modules with a smaller MLEE-style analysis. Its contribution is the verification loop: MLEE's reports require manual confirmation, while KernelLeakGuard confirms or refutes each warning automatically with kernel evidence.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Conclusion

KernelLeakGuard detects memory leaks on Linux kernel early-exit paths with an LLVM-based static analyser and verifies every warning by injecting the predicted failure in a QEMU/KVM guest and checking kmemleak. The project applies core operating-system concepts (kernel memory management, error handling, system calls, per-task kernel state, kernel modules and virtualisation) to a practical systems problem. On a benchmark of 32 modules the static analyser achieves F1 0.94 against 0.73 for a naive missing-kfree analysis, and runtime verification confirms 14 leaks with kernel evidence, refutes the remaining false positive and reaches precision 1.00. The complete tool finds 16 of 17 leaks with no false positive using 425 executions, compared with 15 leaks and 1261 executions for exhaustive fault injection, and generates validated fix patches for 13 of 14 confirmed leaks.

On 39 real Linux modules the static analyser reported 64 warnings; the reviewed ones were false positives caused by patterns it does not yet model, and the runtime stage confirmed none of them, so no false report reached the developer. Exercising the initialisation of real modules under fault injection also exposed a reproducible kernel crash in the null_blk load path and a kmemleak report during erofs initialisation, showing that the runtime half is useful beyond confirming static warnings; both still need root-cause analysis.

Key findings: ownership-transfer analysis is the single most important refinement for precision; predicting the failing call and filtering fault injection to the module's own code make runtime verification cheaper and more precise than blind fault injection; and kmemleak evidence is trustworthy only when stale pointer copies on reused stacks and in reused slab memory are eliminated.

5.2 Future Work

- Field-sensitive heap ownership to detect nested leaks such as klg_l16.
- Model reference-counted lookups (functions returning a put-released object) and error codes stored in variables: the remaining false-positive causes on real kernel code (Section 4.4); the ERR_PTR pattern is already handled.

- Run the runtime stage on all 39 real modules with hardware acceleration (only a representative subset ran, under software emulation), and analyse the root cause of the null_blk crash and check it against the latest upstream kernel before reporting it.
- Reproducing known MLEE-reported leaks on the corresponding historical kernel versions.
- Value-range reasoning about loop counters to avoid rollback-loop false positives.
- Patches for shared error labels (goto ladders used by several paths), which are currently left for manual review.
- Integration into kernel continuous-integration systems such as KernelCI.

Real-world deployment scope. The command-line tool and dashboard can be run by driver developers before submitting patches, and the verifier's evidence (failing call, fault number, kmemleak backtrace) is directly usable in a bug report.

REFERENCES

Wang, Wenwen. “MLEE: Effective Detection of Memory Leaks on Early-Exit Paths in OS Kernels.” 2021 USENIX Annual Technical Conference (USENIX ATC 21), USENIX Association, 2021, www.usenix.org/conference/atc21/presentation/wang-wenwen.

Saha, Suman, et al. “Hector: Detecting Resource-Release Omission Faults in Error-Handling Code for Systems Software.” 2013 43rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), IEEE, 2013.

Emamdoost, Navid, et al. “Detecting Kernel Memory Leaks in Specialized Modules with Ownership Reasoning.” Proceedings of the Network and Distributed System Security Symposium (NDSS), 2021.

Wu, Qiushi, et al. “Understanding and Detecting Disordered Error Handling with Precise Function Pairing.” 30th USENIX Security Symposium (USENIX Security 21), 2021.

Jiang, Zu-Ming, et al. “Fuzzing Error Handling Code Using Context-Sensitive Software Fault Injection.” 29th USENIX Security Symposium (USENIX Security 20), 2020.

Lawall, Julia, and Gilles Muller. “Coccinelle: 10 Years of Automated Evolution in the Linux Kernel.” 2018 USENIX Annual Technical Conference (USENIX ATC 18), 2018.

Palix, Nicolas, et al. “Faults in Linux 2.6.” ACM Transactions on Computer Systems, vol. 32, no. 2, 2014.

Engler, Dawson, et al. “Bugs as Deviant Behavior: A General Approach to Inferring Errors in Systems Code.” Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP), 2001.

Xie, Yichen, and Alex Aiken. “Context- and Path-Sensitive Memory Leak Detection.” Proceedings of ESEC/FSE 2005, ACM, 2005.

Cherem, Sigmund, Lonnie Princehouse, and Radu Rugina. “Practical Memory Leak Detection Using Guarded Value-Flow Analysis.” Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2007.

Sui, Yulei, Ding Ye, and Jingling Xue. “Static Memory Leak Detection Using Full-Sparse Value-Flow Analysis.” Proceedings of the International Symposium on Software Testing and Analysis (ISSTA), ACM, 2012.

Rubio-González, Cindy, et al. “Error Propagation Analysis for File Systems.” Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2009.

Lattner, Chris, and Vikram Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation.” International Symposium on Code Generation and Optimization (CGO), IEEE, 2004.

Nethercote, Nicholas, and Julian Seward. “Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation.” Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2007.

Serebryany, Konstantin, et al. “AddressSanitizer: A Fast Address Sanity Checker.” 2012 USENIX Annual Technical Conference (USENIX ATC 12), 2012.

Bellard, Fabrice. “QEMU, a Fast and Portable Dynamic Translator.” Proceedings of the USENIX Annual Technical Conference, FREENIX Track, 2005.

Bonwick, Jeff. “The Slab Allocator: An Object-Caching Kernel Memory Allocator.” Proceedings of the USENIX Summer 1994 Technical Conference, 1994.

Google. “syzkaller: An Unsupervised Coverage-Guided Kernel Fuzzer.” GitHub, github.com/google/syzkaller. Accessed 12 Oct. 2026.

The Linux Kernel Documentation. “Kernel Memory Leak Detector.” kernel.org, docs.kernel.org/dev-tools/kmemleak.html. Accessed 12 Oct. 2026.

The Linux Kernel Documentation. “Fault Injection Capabilities Infrastructure.” kernel.org, docs.kernel.org/fault-injection/fault-injection.html. Accessed 12 Oct. 2026.

Love, Robert. Linux Kernel Development. 3rd ed., Addison-Wesley, 2010.

Silberschatz, Abraham, Peter B. Galvin, and Greg Gagne. Operating System Concepts. 10th ed., Wiley, 2018.

Marjamäki, Daniel. “Cppcheck: A Tool for Static C/C++ Code Analysis.” cppcheck.sourceforge.io. Accessed 13 Oct. 2026.

APPENDIX: SOURCE CODE

The complete source code is in the project repository (KernelLeakGuard/). Layout:

Table A.1: Repository structure

Path	Contents	Lines
analyzer/src	klg-analyze static analyser (C++)	1583
runtime	klg-agent fault-injection driver (C)	701
vm	kernel/initramfs build, QEMU launcher, guest init	131
corpus	benchmark modules and harness (C)	1589
klg	orchestrator, verifier, evaluation (Python)	2418
dashboard	Streamlit dashboard (Python)	360

Key excerpt: the path search of the static analyser (analyzer/src/Analyzer.cpp).

```
  void dfs(BasicBlock *BB, BasicBlock::iterator It) {
    if (done())
      return;
    if (++Steps > A.O.MaxStepsPerSite) {
      Exhausted = true;
      return;
    }
    bool SavedRealloc = Reallocated;
    for (; It != BB->end(); ++It) {
      Instruction &I = *It;
      if (&I == &Site)
        Reallocated = true; // a new instance; the old one lives on in memory
      if (const Event *E = FT.eventAt(&I))
        if (!Reallocated || E->ViaContainer) {
          Reallocated = SavedRealloc;
          return; // released or handed over: no leak on this path
        }
      if (const auto *CB = dyn_cast<CallBase>(&I))
        if (CB->doesNotReturn()) {
          Reallocated = SavedRealloc;
          return;
        }
      if (const auto *RI = dyn_cast<ReturnInst>(&I)) {
        recordLeak(*RI);
        Reallocated = SavedRealloc;
        return;
      }
      if (isa<UnreachableInst>(&I)) {
        Reallocated = SavedRealloc;
        return;
      }
    }

    Instruction *T = BB->getTerminator();
    if (auto *BR = dyn_cast<BranchInst>(T); BR && BR->isConditional()) {
      bool Neg;
      CondKey K = normalizeCond(BR->getCondition(), Neg);
      for (unsigned S = 0; S < 2 && !done(); ++S) {
        bool KeyValue = (S == 0) != Neg;
        if (A.O.BranchCorrelation) {
          bool Contradicts = false;
          for (size_t Idx = 0; Idx < Decisions.size(); ++Idx)
            if (Decisions[Idx].Key == K && Decisions[Idx].Value != KeyValue &&
                !isStale(Idx))
              Contradicts = true;
          if (Contradicts)
            continue;
        }
```

```
      if (std::optional<bool> IsNull = nullTest(K, KeyValue))
        if (*IsNull && !Reallocated)
          continue; // the allocation itself failed: nothing to leak
      if (errPtrTest(K, KeyValue) && !Reallocated)
        continue; // tracked pointer is an ERR_PTR here: nothing to free
      const CallBase *Trig = errorTrigger(K, KeyValue);
      Decisions.push_back({K, KeyValue});
      if (Trig) {
        Trigs.push_back(Trig);
        TrigSucc.push_back(BR->getSuccessor(S));
      }
      follow(BR->getSuccessor(S));
      if (Trig) {
        Trigs.pop_back();
        TrigSucc.pop_back();
      }
      Decisions.pop_back();
    }
  } else {
    for (BasicBlock *Succ : successors(BB)) {
      if (done())
        break;
      follow(Succ);
    }
  }
  Reallocated = SavedRealloc;
}
```